

RUSH 01

Guia passo a passo para a Piscine C da 42

Objetivo: construir o teu próprio solver para o problema das torres/skyscrapers 4x4, compreendendo cada peça antes de a implementares.

Como usar este guia

Cada fase termina com um pequeno checkpoint. Não avances só porque o código compila: avança quando conseguires explicar oralmente o que o teu programa está a fazer. As instruções abaixo dão a arquitetura, a ordem de implementação, os testes e as perguntas que deves saber responder, mas não entregam uma solução completa pronta a copiar.

Base do guia: enunciado Rush 01 fornecido por ti, versão 8. O enunciado exige um programa 4x4, compilação com `cc -Wall -Wextra -Werror`, apenas `write`, `malloc` e `free` como funções autorizadas, e saída `Error` em caso de input inválido ou sem solução. (enunciado fornecido, pp. 10-15)

Princípio central

Não tentes resolver o puzzle inteiro de uma vez. Separa o problema em 5 camadas: input -> representação -> regras -> pesquisa -> output.

Mapa do projeto

Usa esta ordem. Se uma fase falhar, volta atrás em vez de acrescentares mais lógica por cima.

Fase 0	Ler o problema e desenhar a solução
Fase 1	Criar a estrutura de ficheiros
Fase 2	Fazer o parser do input
Fase 3	Representar e imprimir a grelha
Fase 4	Implementar as regras locais
Fase 5	Transformar as pistas em constraints
Fase 6	Criar o backtracking
Fase 7	Ligar tudo e procurar a primeira solução
Fase 8	Testar sistematicamente e tratar erros
Fase 9	Norm, refactoring e defesa
Bónus	Generalizar até 9x9

Critério para saber que estás pronto para a avaliação: consegues pegar num input, explicar como é dividido em 16 pistas, explicar como uma candidata é testada numa linha/coluna e explicar porque o backtracking termina quando encontra uma solução ou quando todas as tentativas falham.

Fase 0 - Perceber o problema antes do C

O enunciado diz que tens uma grelha 4x4. Em cada linha e coluna devem aparecer as alturas 1, 2, 3 e 4 exatamente uma vez. As 16 pistas dizem quantas torres ficam visíveis de cada direção. (enunciado fornecido, p. 10-11)

0.1 Desenha a grelha e fixa as direções

No papel, desenha uma matriz 4x4. Identifica explicitamente: top de cada coluna, bottom de cada coluna, left de cada linha e right de cada linha. O objetivo é nunca depender de "acho que este índice é o lado de cima".

```
top1 top2 top3 top4 | | | | +---+---+---+---+ left1 | | | | right1 left2 | | | |
right2 left3 | | | | right3 left4 | | | | right4 +---+---+---+---+ | | | | bottom1
... bottom4
```

0.2 Aprende a contar visibilidade

Para uma sequência vista da esquerda, começa com **maior_vista = 0**. Percorre as quatro alturas da esquerda para a direita. Sempre que encontras uma altura maior do que **maior_vista**, essa torre fica visível e atualizas o maior valor. No fim, o número de atualizações é a visibilidade.

Faz 5 sequências manualmente no papel antes de escrever código. Inclui uma sequência crescente, uma decrescente, uma em que o 4 aparece a meio e outra em que há números pequenos escondidos atrás de maiores.

0.3 Descobre as consequências óbvias

Pensa nas pistas extremas. Uma pista igual a 1 diz-te que, desse lado, a primeira torre visível tem de ser 4. Uma pista igual a 4 implica uma ordem totalmente crescente a partir daquele lado. Escreve estas consequências em frases simples antes de programar.

Checkpoint 0

- ☐ Consigo calcular a visibilidade de qualquer sequência de 4 números sem hesitar.
- ☐ Consigo dizer a que pista corresponde cada lado da grelha.
- ☐ Consigo explicar o que significa "uma de cada altura por linha e coluna".

Fase 1 - Criar a estrutura do projeto

A entrega deve conter todos os ficheiros .c necessários para compilar o executável rush-01. O enunciado não obriga a uma organização específica, por isso podes escolher uma estrutura pequena e coerente. (enunciado fornecido, p. 10)

1.1 Organização sugerida

```
ex00/ ■■■ main.c ■■■ input.c ■■■ solver.c ■■■ checks.c ■■■ print.c ■■■ rush01.h
```

Esta é apenas uma proposta. Não precisas de copiar esta organização da referência. O importante é cada ficheiro ter uma responsabilidade óbvia.

1.2 Decide os dados

Antes das funções, escolhe estas duas estruturas conceptuais: **grid** para os 16 valores da solução e **clues** para as 16 pistas. Para a versão obrigatória, uma matriz 4x4 é suficiente para a grelha e um array de 16 inteiros é suficiente para as pistas.

Decide também como sinalizar uma célula vazia. Uma convenção simples é usar 0 como "ainda não preenchida". Depois escreve no cabeçalho apenas as assinaturas das funções que pretendes criar. Exemplo de assinatura, não implementação:

```
int parse_input(char *str, int *clues); int solve(int grid[4][4], int clues[16], int pos);  
int is_valid(...); int visible_count(...); void print_grid(int grid[4][4]);
```

1.3 Compila imediatamente

```
cc -Wall -Wextra -Werror -o rush-01 *.c
```

Neste momento é aceitável ter funções vazias e temporárias, desde que o teu projeto compile. O objetivo é eliminar cedo problemas de include, nomes e ligação entre ficheiros.

Checkpoint 1

- ☐ A árvore de ficheiros está criada.
- ☐ O header contém os protótipos que precisas.
- ☐ O comando de compilação do enunciado funciona sem warnings.

Fase 2 - Parser: transformar argc/argv em 16 pistas

O programa recebe uma única string com 16 valores separados por espaços, no formato indicado pelo enunciado. Qualquer outro input deve ser considerado erro. (enunciado fornecido, p. 11)

2.1 Trabalha em duas perguntas simples

Pergunta A: a forma do input está correta? **Pergunta B:** os 16 valores são inteiros entre 1 e 4? Primeiro garante a forma, depois o conteúdo.

2.2 Implementa o parser por etapas

1. Confirma `argc == 2`. 2. Percorre a string com um índice. 3. Sempre que encontra um dígito válido, converte-o para inteiro. 4. Exige um separador apropriado entre valores. 5. No final, garante que obtiveste exatamente 16 valores e que não sobraram caracteres inválidos.

Não uses ainda a lógica do puzzle para validar o input. Parser e solver são problemas diferentes.

2.3 Testes mínimos do parser

- ☐ Input com exatamente 16 números válidos -> aceitar.
- ☐ Input com 15 números -> rejeitar.
- ☐ Input com 17 números -> rejeitar.
- ☐ Um valor 0 -> rejeitar.
- ☐ Um valor 5 -> rejeitar.
- ☐ Letras no meio -> rejeitar.
- ☐ String vazia ou argumento em falta -> rejeitar.
- ☐ Separadores inesperados -> rejeitar conforme as regras do enunciado.

2.4 Onde podes simplificar

Para o problema obrigatório, só precisas de reconhecer números de 1 a 4. Isso significa que não precisas de construir um parser decimal genérico de vários dígitos. Mantém o parser pequeno e fácil de defender oralmente.

Checkpoint 2

- ☐ Recebo uma string e consigo obter `clues[0] ... clues[15]`.
- ☐ Inputs inválidos saem com `Error` e `newline`.
- ☐ Nenhuma parte do solver depende de strings.

Fase 3 - Grelha e impressão

Agora cria uma forma previsível de testar o resto do programa sem ainda teres de resolver puzzles.

3.1 Imprime uma matriz fixa

Inicializa uma grelha de teste manual, por exemplo uma solução conhecida do próprio enunciado, e cria `print_grid()`. A saída deve ter os quatro números de cada linha separados por espaços e uma quebra de linha entre linhas. O formato esperado está explicitado no enunciado. (enunciado fornecido, p. 11)

3.2 Mantém print isolado

O resto do programa não deve saber como se imprime. Isso facilita mudar a formatação sem tocar no solver.

Checkpoint 3

- [] Consigo imprimir qualquer matriz 4x4 no formato exato.
- [] Consigo trocar a matriz de teste sem mexer em `print_grid()`.

Fase 4 - As regras que uma candidata precisa de respeitar

Esta é a fase mais importante antes do backtracking. O solver só fica simples quando tens uma função de validação fiável.

4.1 Regra 1: valores 1..4

Qualquer valor colocado na grelha deve estar entre 1 e 4. Na geração de candidatos vais cumprir isto por construção, mas a função de validação pode continuar a assumir esta pré-condição.

4.2 Regra 2: não repetir na linha

Antes de colocares um candidato numa célula, percorre as células já preenchidas da mesma linha. Se o número já aparecer, essa tentativa é inválida.

4.3 Regra 3: não repetir na coluna

Faz exatamente a mesma ideia na coluna. Isto garante que, quando uma linha ficar completa, terá os quatro valores diferentes.

4.4 Regra 4: visibility constraints

A grande decisão: **não precisas de exigir que uma linha parcial já tenha a visibilidade final exata**. Em vez disso, verifica se o prefixo ainda pode vir a tornar-se compatível com a pista. Uma implementação inicial mais simples é validar a pista apenas quando a linha/coluna estiver completa. Depois podes acrescentar podas mais fortes para acelerar.

4.5 Função de visibilidade

Cria uma função conceptual que receba uma sequência de 4 valores e conte quantas torres são recordes sucessivos. Depois cria uma variante que percorra a mesma sequência ao contrário para a vista oposta.

```
visible_left(line) visible_right(line) visible_top(column) visible_bottom(column)
```

Não copies quatro funções quase iguais. Pensa em uma função genérica que recebe a sequência e o sentido, ou em uma pequena função auxiliar que constrói a ordem de leitura que precisas.

Checkpoint 4

- ☐ Consigo rejeitar uma repetição numa linha antes de continuar.
- ☐ Consigo rejeitar uma repetição numa coluna.
- ☐ Consigo calcular visibilidade dos dois lados da mesma linha.
- ☐ Consigo explicar por que uma pista de 1 força um 4 na primeira posição vista.

Fase 5 - Transformar pistas em informação útil

Agora usa as clues para reduzir o espaço de pesquisa. Não dependas apenas de brute force cego; aproveita as regras simples que o puzzle te dá.

5.1 Casos especiais que deves detetar cedo

Pista 1: a primeira posição do lado dessa pista tem de ser 4. Pista 4: as quatro posições, a partir desse lado, têm de subir 1-2-3-4. Casos semelhantes podem resultar da combinação de duas pistas opostas.

5.2 Pré-calcular as permutações

Uma linha completa válida é uma permutação de 1,2,3,4. Existem apenas $4! = 24$ sequências possíveis. Isto é uma enorme simplificação: em vez de inventares números arbitrários, podes gerar ou listar conceptualmente as 24 ordens possíveis e filtrar pelas clues.

Escolhe uma das duas abordagens:

A. Gerar candidatos durante o solver: tenta 1,2,3,4 recursivamente por célula e corta repetições.

B. Trabalhar por permutações de linha: cria uma pequena lista das 24 linhas possíveis e, para cada linha, verifica se as pistas left/right dessa linha combinam. Depois o solver combina quatro linhas sem repetir colunas.

Para um primeiro solver, a opção A é mais simples de programar. A opção B é muito boa para entenderes o problema matematicamente e pode tornar o pruning mais forte.

5.3 Cria uma função que responde "ainda é possível?"

Esta é a pergunta certa para o backtracking. Não perguntes apenas "está correto?". Pergunta "com o que já preenchi, ainda existe alguma forma de terminar corretamente?". No mínimo, a tua função pode verificar duplicados e, quando uma linha/coluna fica completa, verificar visibility.

Checkpoint 5

- ☐ Sei explicar porque $4! = 24$ reduz bastante o problema.
- ☐ Consigo filtrar uma linha candidata usando as duas pistas horizontais.
- ☐ Sei distinguir "inválido agora" de "ainda incompleto".

Fase 6 - Construir o backtracking

Aqui está o coração do Rush 01. A ideia é preencher uma célula, verificar se a escolha ainda pode funcionar e, se mais tarde descobrires que não funciona, desfazes a escolha e tentas outra.

6.1 Antes do código, escreve este algoritmo no papel

```
solve(position): se position == 16: verificar todas as clues finais retornar
sucesso/fracasso descobrir linha e coluna da position para candidato de 1 a 4: se candidato
viola uma regra local: continuar colocar candidato se o estado ainda pode funcionar: se
solve(position + 1) encontrou solução: retornar sucesso desfazer candidato retornar
fracasso
```

Isto é **pseudocódigo**, não a implementação final. O teu trabalho agora é transformar cada frase numa pequena decisão em C.

6.2 Decide a ordem das células

A opção natural é linha a linha, da esquerda para a direita. Assim, quando acabas uma linha podes validar imediatamente as pistas esquerda/direita dessa linha. Mais tarde podes testar outras ordens se quiseres melhorar o pruning.

6.3 Decide quando validar

Começa com um solver correto e simples: valida duplicados imediatamente; valida visibility horizontal quando uma linha fica completa; valida visibility vertical quando uma coluna fica completa. Só depois tenta podas parciais.

6.4 O momento do "desfazer"

Este é o erro clássico. Sempre que uma tentativa falhar, a célula que acabaste de preencher tem de voltar ao estado vazio antes de tentares o candidato seguinte ou regressares para trás. Testa isto imprimindo temporariamente as posições durante a pesquisa.

6.5 Parar na primeira solução

O enunciado pede a primeira solução encontrada caso existam várias. Portanto, quando `solve()` chegar ao fim com sucesso, propaga imediatamente o sucesso para cima e não continues a procurar.

Checkpoint 6

- ☐ Consigo explicar recursion sem dizer apenas "é magia".
- ☐ Se uma tentativa falha, os valores temporários são desfeitos.
- ☐ Quando a solução é encontrada, o programa para a pesquisa.

Fase 7 - Ligar tudo no main

7.1 Fluxo final

```
main -> validar argc -> parse_input -> inicializar grid a zeros -> solve -> se sucesso:  
print_grid -> se fracasso: print Error
```

Mantém o main pequeno. O main deve coordenar, não conter o algoritmo inteiro.

7.2 Teste oficial do enunciado

```
./rush-01 "4 3 2 1 1 2 2 2 4 3 2 1 1 2 2 2"
```

Compara a saída com o exemplo fornecido pelo próprio enunciado. (enunciado fornecido, p. 11)

7.3 Testes que tens de inventar

Não testes apenas um puzzle conhecido. Cria inputs que incluam: uma solução fácil com muitas pistas extremas; um puzzle válido com menos pistas extremas; um input impossível; inputs estruturalmente inválidos; e um puzzle com várias soluções para verificares que o programa simplesmente escolhe a primeira que encontra.

7.4 Debug visual

Quando estiveres bloqueado, usa um modo temporário em que o solver imprime a grelha cada vez que coloca uma célula. Quando entenderes o erro, remove esses prints. O objetivo é perceber a árvore de pesquisa e descobrir em que ponto uma linha/coluna se torna impossível.

Checkpoint 7

- [] Input válido conhecido produz a grelha esperada.
- [] Input impossível produz apenas Error.
- [] Input inválido produz apenas Error.
- [] Sem warnings com -Wall -Wextra -Werror.

Fase 8 - Testes sistemáticos e robustez

O enunciado exige tratamento coerente de erros e lembra que uma falha de compilação dá 0. (enunciado fornecido, pp. 5-6)

8.1 Testes de input

- ☐ `argc = 1`
- ☐ `argc > 2`
- ☐ menos de 16 valores
- ☐ mais de 16 valores
- ☐ valor 0
- ☐ valor 5
- ☐ caracteres não numéricos
- ☐ separadores estranhos
- ☐ string vazia

8.2 Testes das regras

- ☐ linha com repetição
- ☐ coluna com repetição
- ☐ visibilidade esquerda errada
- ☐ visibilidade direita errada
- ☐ visibilidade top errada
- ☐ visibilidade bottom errada

8.3 Testa os helpers isoladamente

Antes de culpare o backtracking, testa `visible_count`, teste de linha, teste de coluna e parser com pequenos casos conhecidos. Um solver recursivo em cima de uma função de visibilidade errada pode ser muito difícil de depurar.

8.4 Performance sem obsessão

4x4 é pequeno. Primeiro procura correção. Depois conta quantas chamadas recursivas fazes em inputs difíceis. Se forem muitas, reforça o pruning em vez de reescrever tudo.

Checkpoint 8

- ☐ Tenho testes para parser, regras e solver separadamente.
- ☐ Sei reproduzir cada bug importante antes de o corrigir.
- ☐ Removi prints de debug do resultado final.

Fase 9 - Preparar Norm e defesa

A avaliação exige que todos os elementos da equipa conheçam os detalhes do projeto e saibam explicá-los. O enunciado também salienta a importância da conformidade com a Norm e da compilação correta. (enunciado fornecido, pp. 5-6)

9.1 Refactor final

- ☐ Cada função tem uma responsabilidade clara.
- ☐ Não tens código duplicado sem necessidade.
- ☐ Nomes de variáveis explicam o que representam.
- ☐ Não tens funções gigantes.
- ☐ Os includes estão no header correto.
- ☐ O projeto compila com exatamente as flags do enunciado.
- ☐ Só entregas os ficheiros necessários.

9.2 Perguntas que tens de conseguir responder

- ☐ Porque é que o input tem 16 números?
- ☐ Como transformas argc/argv nas pistas?
- ☐ Como calculas a visibilidade de uma sequência?
- ☐ Porque é que uma pista 1 obriga a existir um 4 no extremo?
- ☐ Porque não podes repetir um número numa coluna?
- ☐ O que é backtracking?
- ☐ Em que momento desfazes uma escolha?
- ☐ Como é que sabes que uma solução encontrada é realmente válida?
- ☐ Porque é que o programa pode parar na primeira solução?
- ☐ Porque é útil separar parser, checks, solver e print?

Quando conseguires responder a estas perguntas sem olhar para o código, já não estás apenas a "ter um programa que funciona". Estás a ter um projeto que consegues defender.

BÓNUS - Generalizar o Rush 01 até 9x9

O enunciado permite, como bónus, lidar com outros tamanhos de mapa até 9x9. Isto é opcional e deve ser feito apenas depois da versão obrigatória estar sólida. O próprio enunciado avisa que um bónus não compensa falhas na parte obrigatória. (enunciado fornecido, p. 11)

B.1 Remove o "4" espalhado pelo código

Cria uma variável `size`. A grelha passa a ser `size x size`. Os valores possíveis deixam de ser 1..4 e passam a ser 1..`size`. As 16 clues passam a ser `4*size`.

B.2 O parser precisa de números maiores

Aqui já tens de aceitar valores como 5, 6, 7, 8 e 9, portanto o parser deixa de poder assumir um único dígito. Reaproveita a ideia de "ler dígitos até ao separador e construir o inteiro".

B.3 A função de visibilidade já não é específica do 4

A lógica é exatamente a mesma: percorre `size` posições e conta novos máximos. Não cries uma função nova por tamanho.

B.4 O espaço de pesquisa explode

Aqui o brute force simples fica muito mais pesado. O pruning torna-se central. A ideia das permutações continua válida, mas o número de permutações cresce como $n!$, por isso precisas de filtrar candidatos por clues e por colunas o mais cedo possível.

B.5 Ordem de implementação do bónus

- ☐ parametrizar grid e clues pelo tamanho
- ☐ generalizar parser
- ☐ generalizar print
- ☐ generalizar duplicate checks
- ☐ generalizar visibility
- ☐ testar 5x5 com um caso pequeno
- ☐ só depois pensar em otimização
- ☐ validar novamente a versão 4x4 depois de cada alteração

B.6 Desafio extra para aprenderes mais

Em vez de copiares uma solução maior, mede o número de estados pesquisados para 4x4, 5x5 e 6x6. Depois tenta reduzir esse número através de melhores constraints. Este exercício ensina-te mais sobre algoritmos do que simplesmente fazer o programa "funcionar".

Checklist final antes de entregar

- ☐ A pasta ex00/ contém todos os .c e headers necessários.
- ☐ Compila com `cc -Wall -Wextra -Werror -o rush-01 *.c`.
- ☐ Input correto -> programa encontra e imprime uma solução.
- ☐ Input inválido -> apenas Error + newline.
- ☐ Input sem solução -> apenas Error + newline.
- ☐ Não há prints de debug.
- ☐ A grelha respeita unicidade nas linhas e colunas.
- ☐ As 16 clues são respeitadas.
- ☐ O código está limpo e organizado.
- ☐ Consigo explicar cada função da equipa.
- ☐ O bônus, se feito, não partiu a versão obrigatória.

Regra de ouro: quando ficares bloqueado, reduz o problema até encontrares uma função que consigas testar isoladamente. Se o solver falha, não assumes que o backtracking está errado. Primeiro verifica parser -> dados -> visibilidade -> validação -> backtracking.

Referência de contexto: o repositório indicado por ti é um repositório público de material da 42 Piscine que inclui uma pasta Rush01 e apresenta explicitamente a ideia de usar referências para aprender em vez de simplesmente copiar. (README do repositório indicado, consultado online)